

CEFR English Practice Partner: System Architecture & Technical Documentation

1. Project Overview & Unique Selling Proposition

The **CEFR English Practice Partner** is an AI-driven educational platform designed to give ESL (English as a Second Language) students real-time, pedagogically accurate speaking practice.

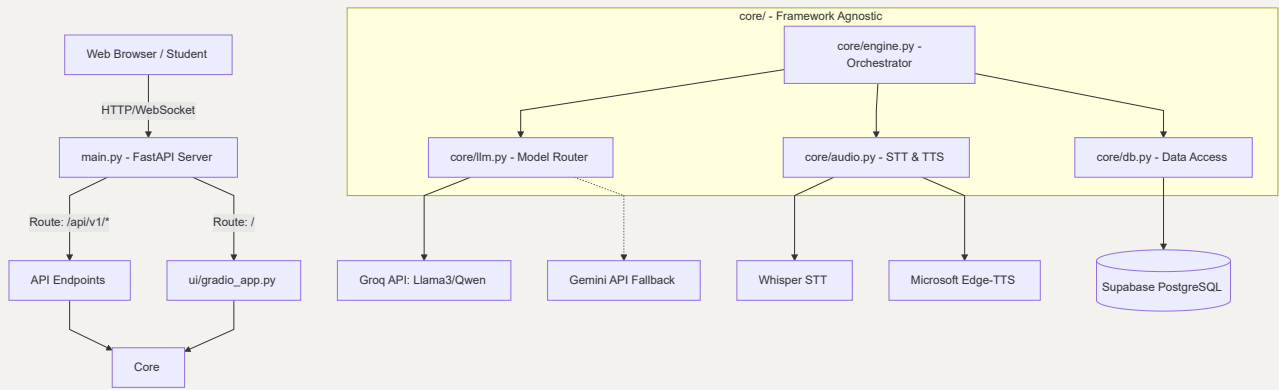
While most AI language apps act as unstructured chatbots, our system acts as both a **conversational partner** and a **strict CELTA-qualified assessor**.

Our Unique Selling Proposition (USP): We provide instantaneous, CEFR-calibrated spoken conversation alongside rigorous, 11-category grammatical feedback that prevents AI hallucinations by mandating verbatim quotes from the student's speech.

2. System Architecture: The "Strangler-Fig" Pattern

The application is currently undergoing a "strangler-fig" migration. It was originally built as a monolithic Gradio application, but is now transitioning to a decoupled, API-first architecture to support WebRTC (real-time voice) and multi-tenant classrooms.

Currently, **FastAPI** owns the process. The legacy Gradio UI is mounted as a "leaf" route (`/`), while all business logic has been safely extracted to a framework-agnostic `core/` package.



Technology Stack

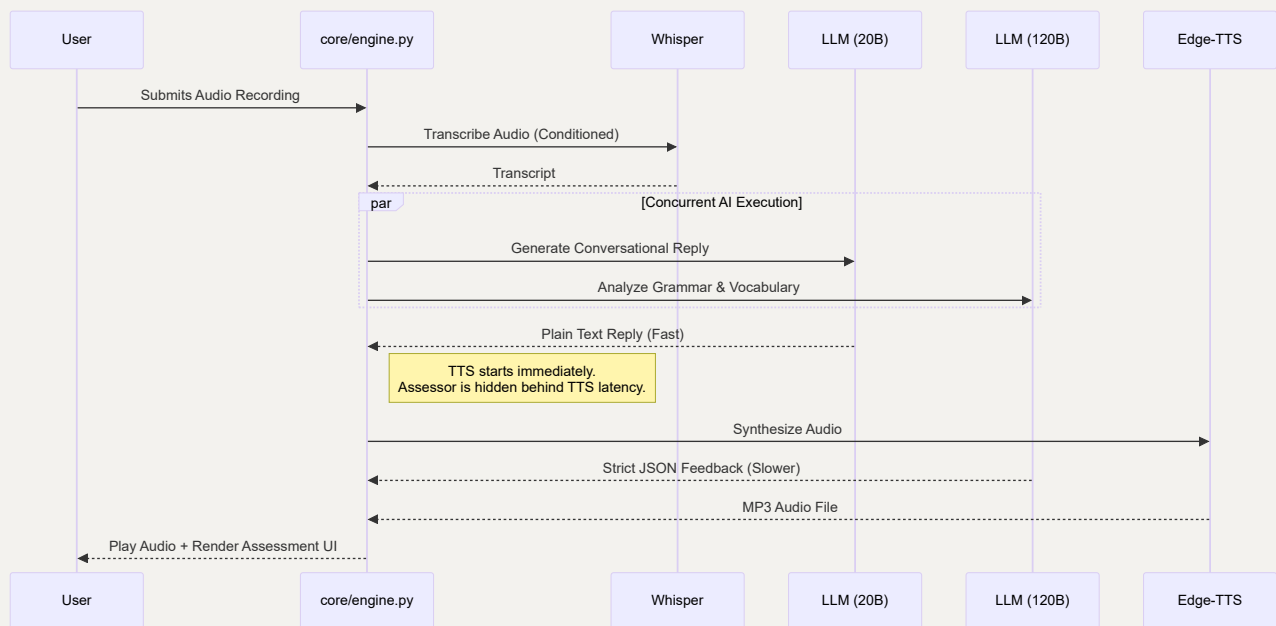
- **Backend Framework:** FastAPI (Python)
- **Legacy UI:** Gradio (Blocks)
- **Database:** Supabase (PostgreSQL)
- **Primary AI Inference:** Groq (gpt-oss-20b for conversation, gpt-oss-120b for assessment)
- **Fallback AI Inference:** Google Gemini (gemini-flash-latest)
- **Speech-to-Text (STT):** Groq Whisper (whisper-large-v3-turbo)
- **Text-to-Speech (TTS):** Microsoft Edge-TTS (via `edge-tts` package)

3. The Pedagogical Engine (`core/engine.py`)

The engine's primary achievement is **zero perceived latency for complex assessment**.

Because a heavy LLM takes time to perform structural grammar analysis, we run the fast conversational reply (`temperature=0.7`) and the heavy analytical assessment (`temperature=0.15`) concurrently.

The heavy assessment finishes in the background while the Text-to-Speech engine is synthesizing the partner's audio.



Key Pedagogical Guardrails

To prevent the "Weak AI" problem (where AI ignores mistakes or hallucinates feedback), the system enforces four strict rules:

1. Whisper Prompt Conditioning

STT systems naturally auto-correct bad grammar. We pass a deliberately flawed prompt:

"Umm, well, he don't like it. I goed to the store..."

to Whisper to force it to output the student's morphological errors verbatim.

2. Verbatim Quotation Guardrail

The Assessor LLM is strictly instructed:

If it cannot quote the student's error character-for-character from the transcript, the error is discarded.

It cannot invent mistakes.

3. Closed Error Taxonomy

The model does not freely guess what is wrong.

It evaluates every clause against exactly **11 hardcoded categories**, such as:

- `verb_tense`
- `preposition`
- `word_order`
- and other predefined grammatical categories.

4. Guided Discovery vs. Explicit Correction

For **A1-A2 learners**, the AI explicitly states the rule.

For **B1+ learners**, the AI uses **Guided Discovery** — asking a concept-checking question and hiding the direct answer in a dropdown to force the student to think.

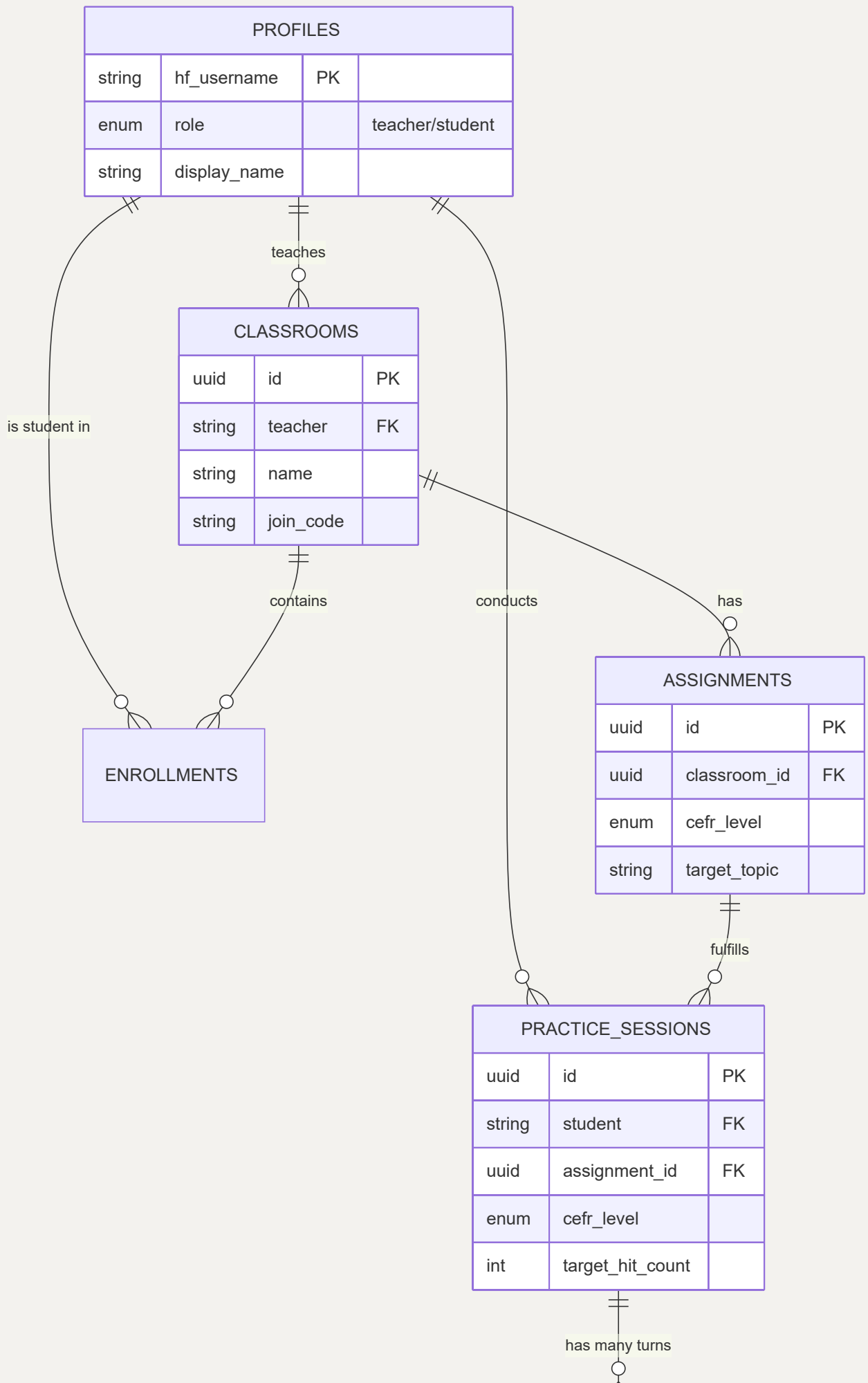
4. Multi-Tenant Classroom Database (`core/db.py`)

The application uses a relational schema in Supabase to support individual learners as well as structured, multi-tenant classrooms.

Security Note

Because we use **Hugging Face OAuth** for identity rather than Supabase JWTs, Row Level Security (RLS) is set to "**deny-all**".

Tenancy boundaries and Teacher-role authorizations are strictly enforced at the application level in `core/db.py`.



(1)

EXCHANGES			
bigint	id	PK	
uuid	session_id	FK	
int	turn_index		
string	student_text		
jsonb	errors		Validated 11-category JSON

5. Future Scalability & Feature Roadmap

Our current API-first architecture, decoupled pedagogical engine, and structured database provide the foundation to scale this platform significantly.

The following features are the next logical steps for expansion:

1. Comprehensive AI Conversational Partner (Real-time)

By migrating from Gradio to a custom React frontend connected to FastAPI via WebRTC/WebSockets, we will shift from **"Walkie-Talkie" style practice** to fully interruptible, real-time voice conversations.

2. Multi-Skill Integration (Reading, Writing, Listening)

While Speaking is our **Unique Selling Point (USP)** and core differentiator, the 11-category assessment engine can be directly applied to text inputs.

We can scale to offer:

- Writing evaluations
- Listening comprehension checks
- Reading pronunciation assessments

seamlessly.

3. Automated End-of-Session Teacher Reports

The system currently generates markdown reports for the student.

We will scale this so that once a student finishes an assigned session, a **cohort-aggregated report** — highlighting macro-trends such as:

■ *"60% of your class failed on Prepositions today"*

— is automatically generated and sent directly to the **Teacher's Dashboard**.

4. AI Placement Testing & Final Exams

Using the existing engine, we can build a specialized "**Exam Mode**."

Instead of a conversational partner, the AI will act as a strict examiner following an oral testing rubric.

It will dynamically adapt questions based on previous answers to:

- Pinpoint a student's exact CEFR level upon signup
- Conduct formal final exams
- Certify level completion

5. Multi-Language and Accent Expansion

The core pipeline:

STT → **LLM Router** → **TTS**

is completely language-agnostic.

By simply adjusting the `core/config.py` system prompts and swapping out the Microsoft Edge-TTS voices, the platform can be scaled to support:

- Spanish
- French
- German
- Specific regional English accents

- Australian English comprehension
- British English comprehension

with minimal backend changes.